

RAPORT TECHNICZNY

AGtest-framework v0.1

Analiza struktury projektu, kodu, aktualnej funkcjonalności, wyników weryfikacji oraz rekomendowanego kierunku dalszego rozwoju.

Projekt: pymoo_gui_framework

Typ: desktopowy framework eksperymentów MOEA

Technologie: Python, PyQt5, pymoo, Platypus, NumPy

Wersja: v0.1

Data aktualizacji: 22 września 2026

Zakres: struktura, runtime, GUI, metryki, eksport, testy, EXE

1. Podsumowanie wykonawcze

Framework jest działającą aplikacją eksperymentalną do optymalizacji wielokryterialnej, a nie tylko prostym interfejsem do biblioteki `pymoo`. Ma własne rejestry komponentów, implementacje i adaptory algorytmów, benchmarki, warstwę metryk, eksport XLSX, wykonywanie wsadowe, interaktywną historię epok oraz wizualizację wyników.

17

algorytmów

62

konfiguracje problemów

8

metryk jakości

3

karty GUI

92

testy

580

deklaracji runtime

Ocena: zakres funkcjonalny pozwala prowadzić eksperymenty i przygotowywać wyniki bez pisania osobnego skryptu. Od poprzedniej analizy istotnie rozbudowano sterowanie przebiegiem, eksport grafiki, metrykę Delta, testy GUI oraz proces wydania samodzielnego EXE. Nadal najważniejsze są: zgodność algorytm–problem, problemowe punkty HV, powtórzenia statystyczne i pełne metadane eksperymentu.

2. Struktura projektu

Element	Odpowiedzialność
<code>src/pymoo_gui/app.py</code>	Główne GUI, dynamiczne formularze, walidacja, sterowanie ciągle i krokowe, historia epok, tabele, komunikaty i eksport.
<code>src/pymoo_gui/algorithms</code>	Rejestr algorytmów, implementacje, adaptery oraz wspólny dispatcher <code>minimize</code> .
<code>algorithms/common.py</code>	Callback generacji, normalizacja populacji, wybór rozwiązań dopuszczalnych i niezdominowanych oraz wywołanie metryk.
<code>src/pymoo_gui/problems</code>	Lokalne benchmarki, rejestr problemów, fabryki oraz ładowanie znanych frontów Pareto.
<code>metrics/quality.py</code>	Obliczanie wskaźników jakości, w tym uogólnionej Delta i KKTPM.
<code>metrics/export.py</code>	Generowanie arkuszy XLSX bez zależności od pandas lub openpyxl.
<code>multi.py</code>	Wsadowe wykonywanie iloczynu problemów i algorytmów oraz trwały zapis częściowych wyników.
<code>parallel.py</code>	Opcjonalna ewaluacja osobników przez pulę procesów lub wątków.
<code>viz</code>	Wizualizacja Pareto 1D/2D/3D, siatka, eksport PNG i przebiegi metryk.
<code>scripts</code>	Weryfikacja EXE oraz automatyczne generowanie aktualnego katalogu funkcji.
<code>tests</code>	Testy rejestrów, algorytmów, metryk, GUI, trybu krokowego, wykresów, Multi i ścieżek runtime.

Uruchomienie rozpoczyna się w `run_gui.py` lub przez entry point `AGtest-framework`. `MainWindow` buduje interfejs z rejestrów `PROBLEMS` i `ALGORITHMS`. Obliczenia odbywają się poza wątkiem GUI. Standardowe algorytmy trafiają do `pymoo.optimize.minimize`, natomiast adaptery i algorytmy badawcze mogą udostępniać własne `gui_minimize`.

Historyczny pakiet `pymoo_gui.algoritms` pozostaje aliasem zgodności wstecznej dla kanonicznego `pymoo_gui.algorithms`.

3. Aktualna funkcjonalność

3.1. Karta Main

Karta obsługuje kompletny pojedynczy eksperyment:

- wybór problemu i algorytmu z rejestrów oraz dynamiczne formularze parametrów;
- walidację liczb, wartości logicznych, wyborów i danych ogólnych;
- konfigurację liczby generacji, ziarna, komunikatów i ewaluacji równoległej;
- automatyczny albo ręczny punkt referencyjny hiperobjętości;
- podgląd frontu Pareto, populacji i rozwiązań niezdominowanych;
- tabelę metryk dla generacji, trajektorie, konsolę i automatyczny eksport.

3.2. RAN, RAN step i historia epok ROZBUDOWANO

Tryb **RAN** nie narzuca limitu generacji. Nowy **RAN step** uruchamia obliczenia bez limitu, zatrzymuje worker po pierwszej generacji i wymaga jawnego polecenia kolejnego kroku. Synchronizacja opiera się na warunku wątku, nie kolejkuje wielokrotnych szybkich kliknięć i reaguje na zatrzymanie oraz zamknięcie okna.

Każda migawka wykresu jest kopiowana do historii. Użytkownik może przejść do poprzedniej lub następnej obliczonej epoki, podać numer albo wrócić przez **Latest**. Przeglądanie starej epoki nie zatrzymuje rejestrowania nowych metryk.

3.3. Sterowanie wykresem i eksport PNG NOWE

- przełącznik siatki oraz ręczny krok podziałki z trybem automatycznym;
- większe etykiety osi i wyraźniejsze punkty rozwiązań niezdominowanych;
- zachowanie bieżących zakresów osi oraz kąta widoku 3D;
- **Export PNG** zapisujący kompletny wykres z osiami, legendą i tytułem;
- **Screenshot** zapisujący całe okno, w tym aktywną kartę, ustawienia i konsolę;
- kwadratowy układ startowy i ograniczona wysokość dolnej konsoli.

3.4. Karta Multi

Karta wykonuje iloczyn **wybrane problemy × wybrane algorytmy**. Obsługuje wielokrotny wybór, limit generacji, wspólne ziarno, pasek postępu, status kombinacji, anulowanie i kontynuowanie po błędzie pojedynczego uruchomienia. Wyniki zakończonych lub częściowo wykonanych przebiegów są zachowywane; każda kombinacja otrzymuje osobny plik metryk i rozwiązań. Karta celowo nie renderuje wykresów.

Ograniczenie: Multi nadal używa domyślnych parametrów rejestru i jednego ziarna dla wszystkich kombinacji.

3.5. Karta Metric trajectories

Karta pokazuje osiem osobnych wykresów wartości względem generacji: IGD, GD, IGD+, GD+, Spread, Delta, HV i KKTPM. Brak poprawnej wartości jest oznaczany jako **N/A**. Eksperymenty Multi nie zmieniają tych wykresów. Dla Spread, Delta i KKTPM wyłączono skróty SI, aby prezentować pełne wartości dziesiętne.

3.6. Algorytmy

Zarejestrowane są: AGE-MOEA, Eps-MOEA, Eps-NSGA-II, GDE3, HypE, IBEA, Learning-across-problems EDMO, LMOEA-DS, MOEAD, MOEA/D z rodziny epsilon przez Platypus, NSGA-II, NSGA-III, R-NSGA-III, RVEA, SPEA2, RNN-guided DMO oraz TS-NSGA.

Framework integruje standardowy obiekt pymoo, adapter biblioteki zewnętrznej albo własny algorytm z metodą `gui_minimize`. `empty_algorithm_template.py` pozostaje punktem startowym nowej implementacji.

3.7. Problemy benchmarkowe

Rodzina	Liczba konfiguracji	Zakres
Lokalne	12	Binh2, ConstrEx, FON, Golinski, Kursawe, Osyczka2, Schaffer, Srinivas, Tanaka, Viennet2, Viennet3, Water
ZDT	6	ZDT1–ZDT6
DTLZ	7	DTLZ1–DTLZ7
LZ09	9	LZ09_F1–LZ09_F9
UF	10	UF1–UF10
WFG	18	WFG1–WFG9, każdy osobno w 2D i 3D

Łącznie rejestr udostępnia 62 konfiguracje. Pakiet samodzielny zawiera 49 lokalnych plików `.pf`. `empty_benchmark_template.py` dostarcza klasę zgodną z pymoo, walidację, wektorowe `_evaluate`, fabrykę, generator frontu i metadane formularza.

3.8. Metryki i warunki dostępności

Metryka	Warunki i obecne zachowanie
HV	Wymaga punktu referencyjnego o wymiarze zgodnym z liczbą celów.
GD / IGD	Wymagają znanego frontu Pareto i zgodnej liczby celów.
GD+ / IGD+	Jak wyżej; używają implementacji wskaźników z pymoo.
Spread	Obliczany z bieżącego zbioru celów jako znormalizowana miara odległości punktów.
Delta	Uogólniona dla wszystkich algorytmów i 2, 3, 5 oraz większej liczby celów; uwzględnia brak ekstremów i nierównomierność rozstawu.
KKTPM	Wymaga zmiennych decyzyjnych i problemu; wykorzystuje pochodne analityczne albo różnice skończone.

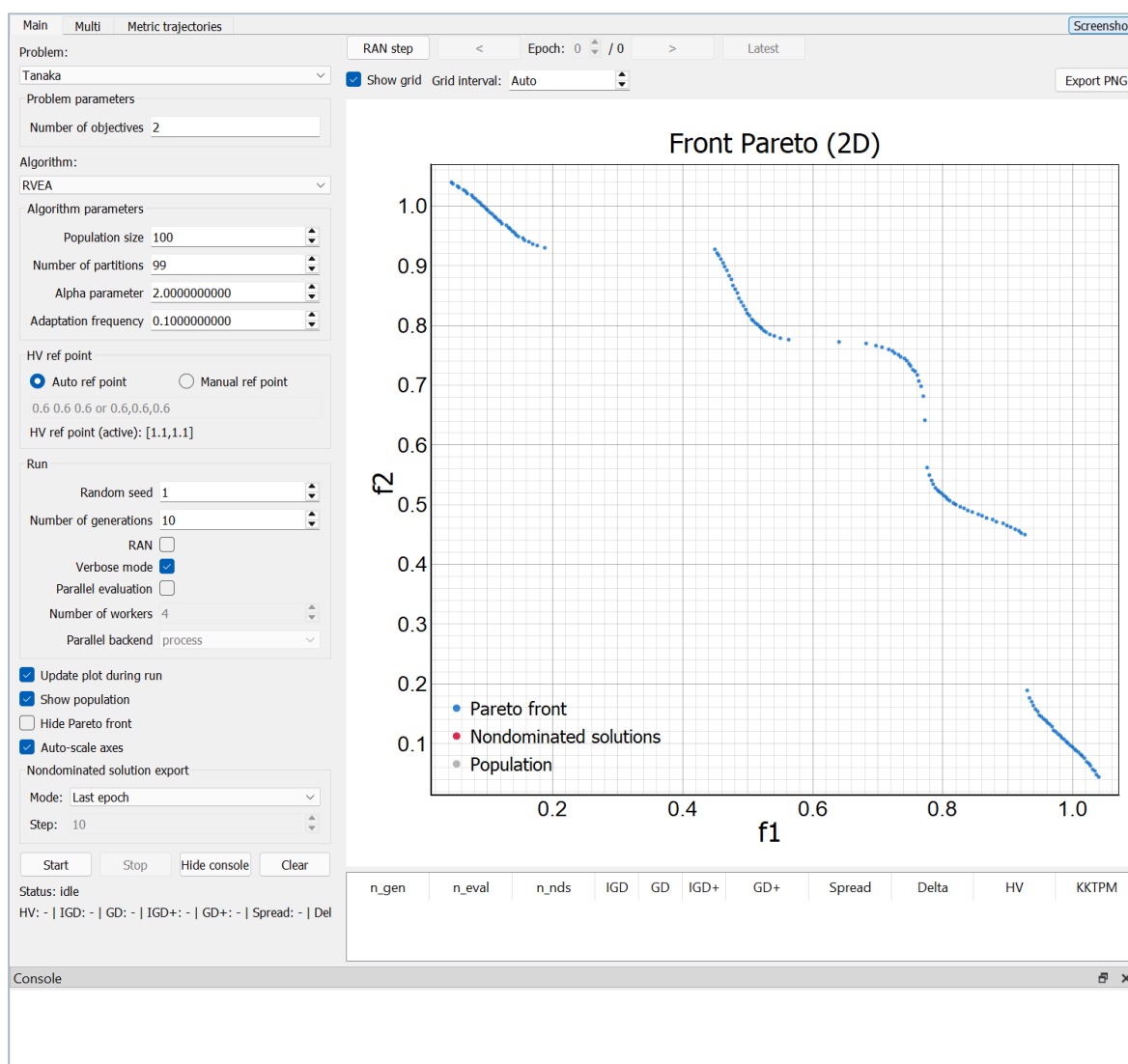
Callback filtruje rozwiązania niedopuszczalne, wyznacza front niezdominowany i dopiero na nim oblicza wskaźniki. Do GUI trafiają także `n_gen`, `n_eval` i `n_nds`.

3.9. Eksport i ścieżki wyników

Framework buduje skoroszyty XLSX bez pandas i openpyxl. Plik metryk zawiera historię generacji, ewaluacji, liczby rozwiązań niezdominowanych i wskaźników. Plik rozwiązań zawiera identyfikator, wartości `f1...fN` oraz — gdy są dostępne — zmienne `x1...xN`. Rozwiązania można zapisywać dla ostatniej generacji, co N generacji albo kaskadowo.

W uruchomieniu źródłowym wyniki trafiają do katalogu projektu. W samodzielnym EXE foldery `metrics_tables` i `solution_tables` są tworzone obok pliku wykonywalnego, niezależnie od katalogu roboczego.

4. Aktualny interfejs v0.1



Rys. 1. Aktualny interfejs: trzy karty, RAN step, historia epok, kontrola siatki, eksport PNG, rozbudowana tabela metryk i przycisk Screenshot.

5. Wyniki weryfikacji

- Wszystkie **92 testy** repozytorium przechodzą.
- Testy obejmują uogólnioną Delta, trajektorie metryk, sterowanie wykresem, RAN step, historię epok, zamykanie zatrzymanego workera, geometrię okna i eksport PNG.
- Samodzielny EXE zweryfikował wszystkie **62 fabryki problemów**.
- W paczce znaleziono i wczytano **49 plików frontów referencyjnych**.
- Wszystkie **17 algorytmów** wykonało krótkie przebiegi na problemie Kursawe.
- Sprawdzono ewaluację thread/process oraz poprawność eksportowanych skoroszytów XLSX.
- Test EXE wykonano poza źródłami, bez Pythona w `PATH` i z zablokowaną siecią.

Wydanie Windows: `AGtest-framework.exe` ma właściwości **FileVersion 0.1**, **ProductVersion 0.1** i opis **AGtest-framework v0.1**. Skrypt wydania generuje raport testu, wersje zależności i SHA-256.

Smoke test potwierdza integrację techniczną, ale nie dowodzi matematycznej identyczności każdej implementacji z publikacją źródłową. Do takiego potwierdzenia nadal potrzebne są testy referencyjne i eksperymenty statystyczne.

6. Problemy i rekomendacje

Priorytet 1 Zgodność algorytm–problem

CZĘŚCIOWO OTWARTE

NSGA-III waliduje obecnie liczbę kierunków względem populacji, co eliminuje część niebezpiecznych konfiguracji. Nadal nie istnieje centralna macierz możliwości dla całego iloczynu Multi. Domyślne punkty R-NSGA-III są 2D, pymoo.MOEAD nie obsługuje każdego problemu z ograniczeniami, a RVEA może wygenerować bardzo dużą liczbę kierunków dla wielu celów.

Należy dodać deklaratywną macierz zgodności, walidację przed startem, wspólny limit kierunków i automatyczne dopasowanie parametrów do `n_obj`.

Priorytet 1 Poprawny punkt referencyjny HV

OTWARTE

Automatyczny punkt jest zasadniczo wektorem 1.1, z wyjątkiem DTLZ1 3D. Nie jest to uniwersalne dla WFG, Kursawe, Water i innych skal. Punkt HV powinien być metadany problemu, wspólny dla porównywanych algorytmów i zapisany wraz z wynikiem.

Priorytet 1 Powtórzenia i statystyka

OTWARTE

Jeden seed nie wystarcza do naukowego porównania metod stochastycznych. Multi powinno przyjmować listę seedów lub liczbę powtórzeń, a raport zbiorczy powinien zawierać średnią, medianę, odchylenie, kwartyle, przedziały ufności, ranking i odpowiednie testy statystyczne.

Priorytet 1

Formalizacja metryk

CZĘŚCIOWO ZREALIZOWANO

Delta ma teraz testy ręcznie obliczalnych przypadków, brakujących ekstremów, nierównomiernego rozstawu, odwróconej kolejności, duplikatów, wartości nieskończoności, wykonalności i wielu wymiarów. Pozostałe wskaźniki nadal wymagają jednolitej dokumentacji wzoru, źródła, kierunku optymalizacji oraz zachowania przy pustym froncie.

Priorytet 2

Testy automatyczne

ISTOTNIE ROZBUDOWANO

Liczba testów wzrosła z 19 do 92. Dodano pokrycie Deltę, sterowania krokowego, historii, eksportu PNG, zakresów osi, zamykania workera, wyglądu okna, ścieżek wyników i samodzielnej paczki. Nadal potrzebne są testy matematycznej zgodności wszystkich algorytmów, pełnej macierzy algorytm-problem i dystrybucji wheel/sdist.

Priorytet 2

Reprodukowalność wyników

OTWARTE

Eksport powinien zawierać arkusz Metadata lub manifest: algorytm, problem, wszystkie parametry, seed, limit, punkt HV, czasy, status, błąd, liczbę ewaluacji, wersje środowiska i identyfikator commita. Obecna konfiguracja nadal nie jest w pełni trwale związana z każdym wynikiem.

Priorytet 2

Podział app.py

OTWARTE

`app.py` ma około 2450 wierszy i łączy GUI, formularze, walidację, wątki, historię epok, prezentację, eksport i tłumaczenie tekstów. Naturalny podział to moduły kart GUI, formularze i walidacja, kontroler uruchomienia, konfiguracja eksperymentu, model wyniku i usługa eksportu.

Priorytet 3

Nazewnictwo i teksty

OTWARTE

W GUI i eksporcie współlistnieją terminy *Generation* i *Epoch*. Warstwa `translate_ui_text` nadal kompensuje historyczne teksty. Docelowo komunikaty powinny być bezpośrednio zapisane poprawnym angielskim UTF-8, a terminologia ujednolicona.

Priorytet 3

Wydajność i odporność zapisu

OTWARTE

- KKTPM z różnicami skończonymi może wymagać wielu dodatkowych ewaluacji.
- Częsty eksport frontów wielokrotnie przepisuje skoroszyt.
- Bezpieczniejszy byłby zapis tymczasowy i atomowa zamiana XLSX.
- Ewaluacja wierszowa może być wolniejsza dla tanich funkcji wektorowych NumPy.
- Zatrzymanie jest kooperacyjne i bardzo długa generacja nie kończy się natychmiast.

7. Zmiany względem poprzedniej analizy

Obszar	Zmiana
Wersja produktu	Jedno źródło wersji 0.1, tytuł v0.1 oraz właściwości Windows EXE.
Sterowanie	RAN step, bezpieczne oczekiwanie workera i przegląd historii epok.
Wykres	Siatka i jej interwał, większe elementy, eksport kompletnego PNG i screenshot okna.
Delta	Obsługa wszystkich algorytmów i wielu wymiarów zamiast ograniczenia do NSGA-II 2D.
Trajektorie	Pełne wartości dziesiętne oraz konsekwentne N/A.
Testy	Wzrost z 19 do 92 oraz test samodzielnego EXE.
Dokumentacja	Aktualny zrzut GUI, generowany katalog funkcji i rozszerzony raport.

8. Rekomendowane przyszłe funkcje

- wykresy metryk względem Function Evaluations;
- porównanie wielu algorytmów na jednym wykresie;
- skala logarytmiczna i wybór widocznych metryk;
- eksport wykresów także do SVG i PDF — PNG jest już dostępny;
- indywidualne parametry algorytmów i problemów w Multi;
- lista seedów, liczba powtórzeń i agregacja statystyczna;
- wznawianie przerwanych eksperymentów;
- zapis i odczyt konfiguracji eksperymentu;
- wybór katalogu wynikowego oraz zbiorczy raport CSV/XLSX/PDF;
- kontrolowane równoległe wykonywanie niezależnych uruchomień Multi;
- parallel coordinates lub scatter-matrix dla więcej niż trzech celów;
- wyszukiwanie list problemów i algorytmów oraz raport kompatybilności przed startem.

9. Zalecana kolejność rozwoju

1. **Macierz zgodności i bezpieczne kierunki referencyjne.**
2. **Poprawne, problemowe punkty HV.**
3. **Wiele seedów, agregacja oraz metadane eksperymentów.**
4. **Testy referencyjne metryk i całej macierzy komponentów.**
5. **Podział app.py i usunięcie powielonej logiki.**
6. **Ujednolicenie terminologii i tekstów.**
7. **Nowe wizualizacje i raporty porównawcze.**

Najważniejsza sekwencja prac: zgodność kombinacji → poprawny HV → powtórzenia i metadane → testy metryk → refaktoryzacja GUI → nowe wizualizacje.

10. Materiały projektu

- `README.md` — instalacja, architektura, przepływy i rozszerzanie frameworka;
- `docs/FUNCTION_CATALOG.md` — generowany katalog aktualnych deklaracji;
- `build/standalone-self-test.json` — raport weryfikacji samodzielnego EXE;
- `src/pymoo_gui/algorithms/empty_algorithm_template.py` — szablon algorytmu;
- `src/pymoo_gui/problems/empty_benchmark_template.py` — szablon benchmarku;
- `docs/AGtest_framework_analysis_PL.html` — edytowalne źródło niniejszego raportu.

Raport powstał na podstawie analizy lokalnego kodu, rejestrów, testów, starej wersji raportu oraz samodzielnego testu dystrybucji Windows. Zachowuje strukturę i charakter wcześniejszego dokumentu, aktualizując go do stanu v0.1 z 22.09.2026.